大数据挖掘技术——k最近邻算法与协同过滤

丁玥
上海交通大学计算机学院
dingyue@sjtu.edu.cn

2026年秋季

饮 水 思 源 • 爱 国 荣 校

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

目 录



1 K最近邻算法（KNN）

2 协同过滤（CF）

3 Spark Mllib示例

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

目 录



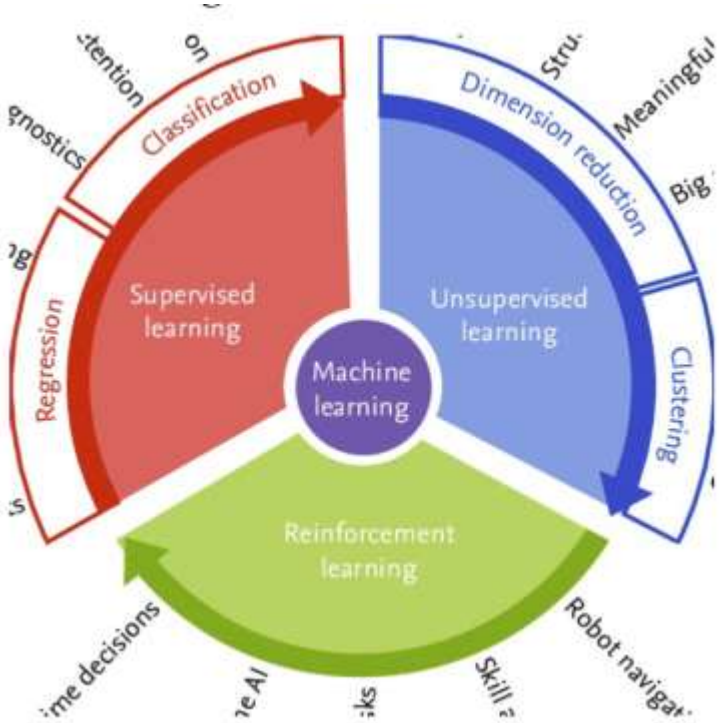
1 K最近邻算法（KNN）

2 协同过滤（CF）

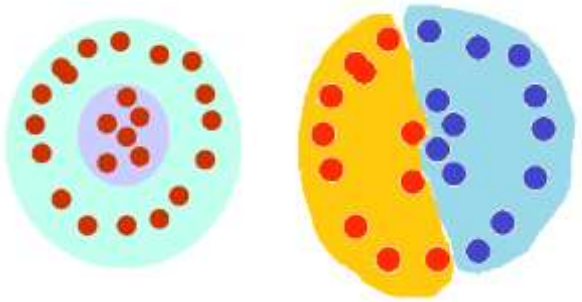
3 Spark Mllib示例

机器学习算法的分类

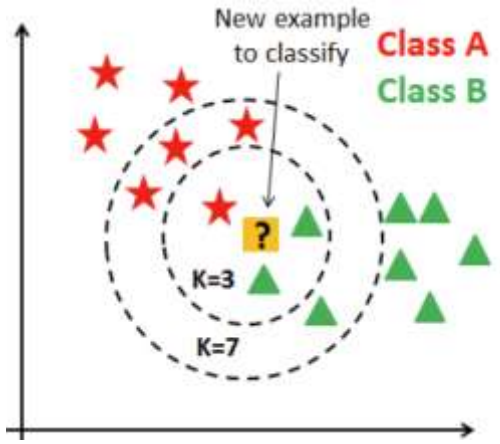
- 有监督学习
- 无监督学习
- 强化学习



K-mean算法属于无监督学习：没有类别的标记信息，从数据本身来学习其中隐含的pattern



K-最近邻算法属于有监督学习:如果一个未知样本最近的K个样本中的大多数属于某一个类别，则该样本也属于这个类别

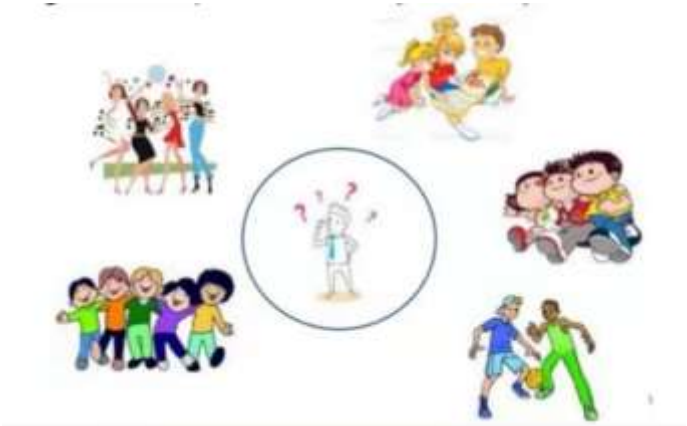


K最近邻算法 (K-Nearest Neighbors, KNN)

- 有监督学习
- 简单但是非常有效的分类算法
- 基于相似性度量进行分类

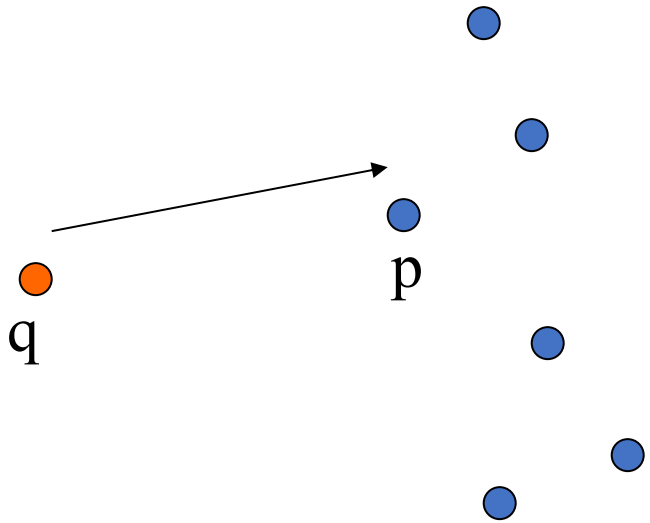
一个简单的类比

- 告诉我你的朋友（你的邻居是谁），然后我就会告诉你你是谁



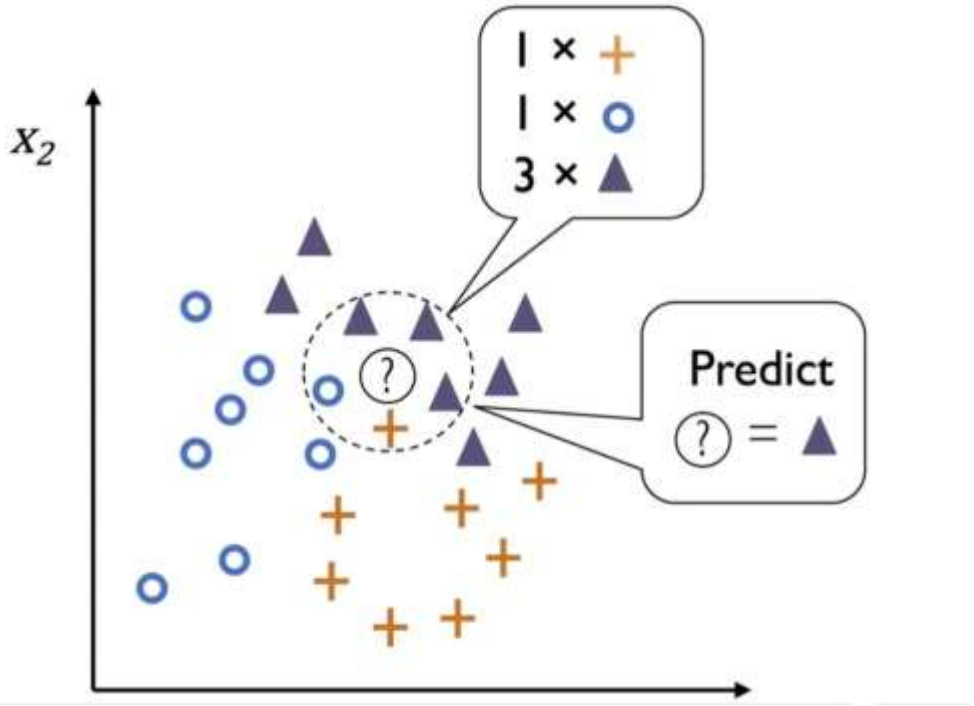
最近邻搜索

- 给定一个集合 P，包含 n 个维度为 d 的数据点
- 目标: 当给定一个查询 q，找到 q 在集合 P 中最相近的 p 个邻居



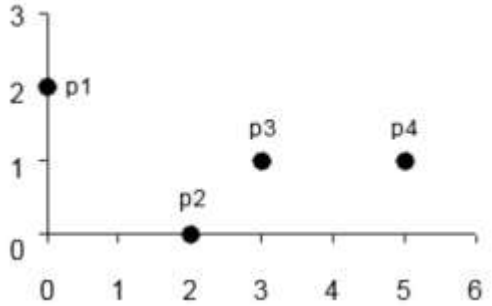
KNN分类

- 根据“**多数投票**”原则对其邻居类别进行分类
  - 找到的**K个最近邻**中最常见的类别(通过测量数据之间的“**距离**”)



- 采用什么样的方式来度量距离（distance measurement）？
- 常见方法：欧式距离（Euclidean Distance）

$$dist(X,Y)=\sqrt{\sum_{i=1}^{n}(x_i-y_i)^2}$$

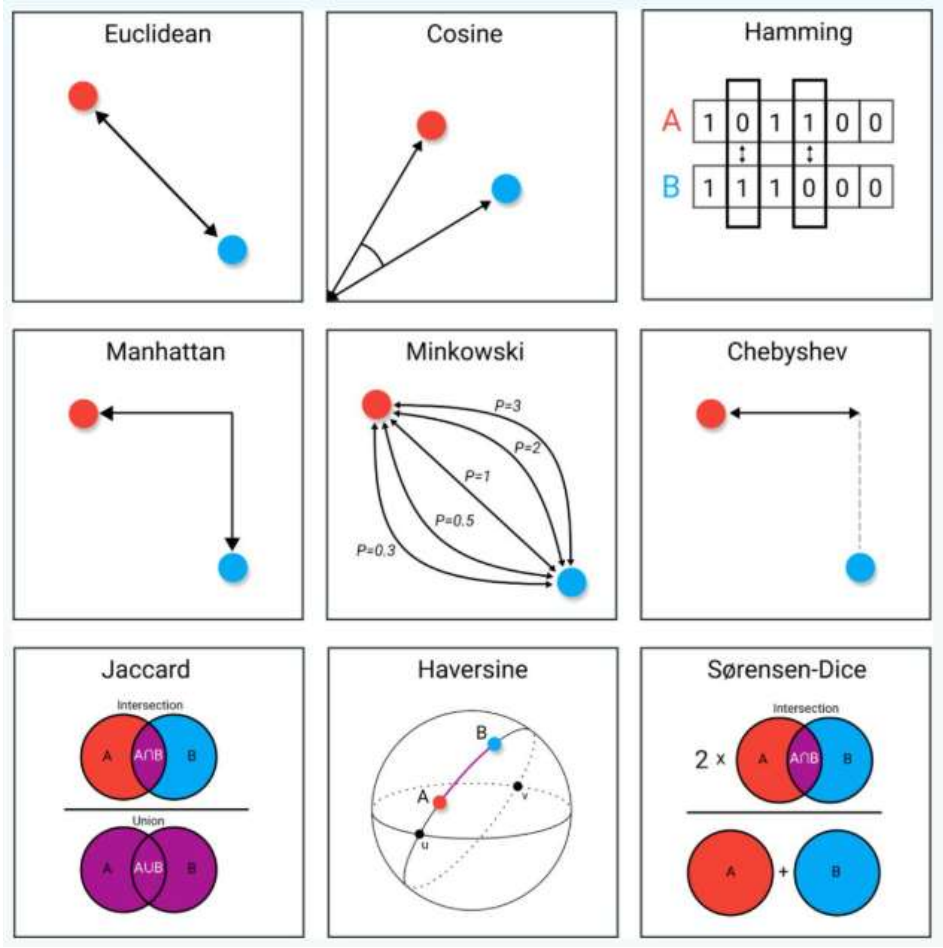


point	x	y
p1	0	2
p2	2	0
p3	3	1
p4	5	1

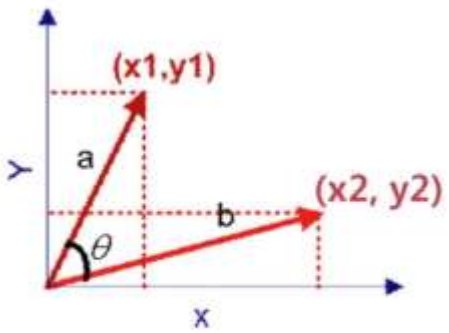
	p1	p2	p3	p4
p1	0	2.828	3.162	5.099
p2	2.828	0	1.414	3.162
p3	3.162	1.414	0	2
p4	5.099	3.162	2	0

- 采用什么样的方式来度量距离（distance measurement）？
- 度量距离的方式有多种
  - 比如Manhattan Distance

$$|x_1 - x_2| + |y_1 - y_2|$$



余弦相似度（Cosine Similarity）



$$\cos(\theta) = \frac{a \bullet b}{||a|| \times ||b||}$$

$$= \frac{(x_1, y_1) \bullet (x_2, y_2)}{\sqrt{x_1^2 + y_1^2} \times \sqrt{x_2^2 + y_2^2}}$$

$$= \frac{x_1 x_2 + y_1 y_2}{\sqrt{x_1^2 + y_1^2} \times \sqrt{x_2^2 + y_2^2}}$$

高维向量计算示例

$\cos(d_1, d_2) = (d_1 \cdot d_2) / ||d_1|| ||d_2||$

d_1= 3 2 0 5 0 0 0 2 0 0

d_2= 1 0 0 0 0 0 0 1 0 2

$$\cos(d_1, d_2) = \begin{cases} 1: \text{exactly the same} \\ 0: \text{orthogonal} \\ -1: \text{exactly opposite} \end{cases}$$

$d_1 \cdot d_2$= 3*1 + 2*0 + 0*0 + 5*0 + 0*0 + 0*0 + 0*0 + 2*1 + 0*0 + 0*2 = 5

$||d_1||$ = (3*3+2*2+0*0+5*5+0*0+0*0+0*0+2*2+0*0+0*0)**0.5**= (42)^**0.5**= 6.481

$||d_1||$ = (1*1+0*0+0*0+0*0+0*0+0*0+0*0+1*1+0*0+2*2)**0.5**= (6)^**0.5**= 2.245

cos(d1, d2) = .3150

K最近邻算法

KNN算法步骤：

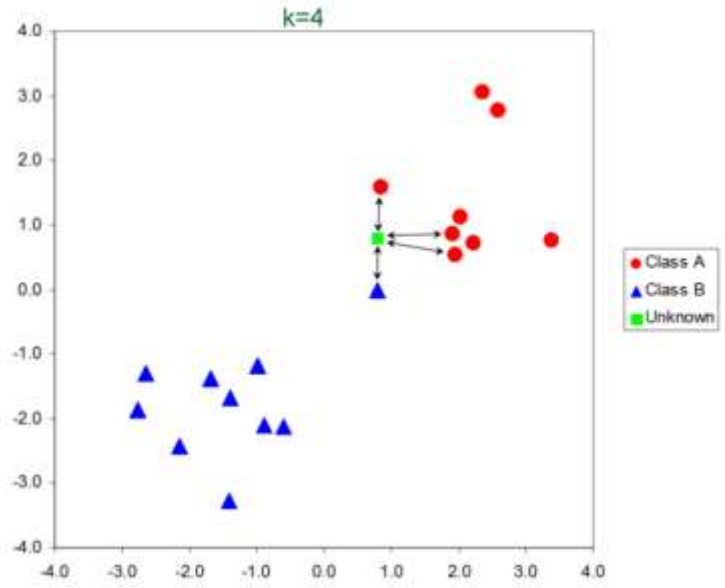
步骤1：确定参数 k = 最近邻的数量

步骤2：计算查询实例与所有训练样本之间的距离

步骤3：根据第 k 个最小距离对距离进行排序，并确定最近邻

步骤4：收集最近邻的类别 Y

步骤5：使用最近邻类别的简单多数作为查询实例的预测值



	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	x_{11}	x_{12}	x_{13}	x_{14}	x_{15}	x_{16}	x_{17}	x_{18}
x_2	1.5																	
x_3	1.4	1.6																
x_4	1.6	1.4	1.3															
x_5	1.7	1.4	1.5	1.5														
x_6	1.3	1.4	1.4	1.5	1.4													
x_7	1.6	1.3	1.4	1.4	1.5	1.8												
x_8	1.5	1.4	1.6	1.3	1.7	1.6	1.4											
x_9	1.4	1.3	1.4	1.5	1.2	1.4	1.3	1.5										
x_{10}	2.3	2.4	2.5	2.3	2.6	2.7	2.8	2.7	3.1									
x_{11}	2.9	2.8	2.9	3.0	2.9	3.1	2.9	3.1	3.0	1.5								
x_{12}	3.2	3.3	3.2	3.1	3.3	3.4	3.3	3.4	3.5	3.3	1.6							
x_{13}	3.3	3.4	3.2	3.2	3.3	3.4	3.2	3.3	3.5	3.6	1.4	1.7						
x_{14}	3.4	3.2	3.5	3.4	3.7	3.5	3.6	3.3	3.5	3.6	1.5	1.8	0.5					
x_{15}	4.2	4.1	4.1	4.1	4.1	4.1	4.1	4.1	4.1	4.1	1.7	1.6	0.3	0.5				
x_{16}	4.1	4.1	4.1	4.1	4.1	4.1	4.1	4.1	4.1	4.1	1.6	1.5	0.4	0.5	0.4			
x_{17}	5.9	6.2	6.2	5.8	6.1	6.0	6.1	5.9	5.8	6.0	2.3	2.3	2.5	2.3	2.4	2.5		
x_{18}	6.1	6.3	6.2	5.8	6.1	6.0	6.1	5.9	5.8	6.0	3.1	2.7	2.6	2.3	2.5	2.6	3.0	
x_{19}	6.0	6.1	6.2	5.8	6.1	6.0	6.1	5.9	5.8	6.0	3.0	2.9	2.7	2.4	2.5	2.8	3.1	0.4

欧几里得距离矩阵D列出了19个样本之间所有可能的成对欧几里得距离，预测X_{11}

一个使用KNN进行预测的例子：从约会网站数据中进行匹配

数据

1	40920	8.326976	0.953952	largeDoses
2	14488	7.153469	1.673904	smallDoses
3	26052	1.441871	0.805124	didntLike
4	75136	13.147394	0.428964	didntLike
5	38344	1.669788	0.134296	didntLike
6	72993	10.141740	1.032955	didntLike
7	35948	6.830792	1.213192	largeDoses
8	42666	13.276369	0.543880	largeDoses
9	67497	8.631577	0.749278	didntLike
10	35483	12.273169	1.508053	largeDoses
11	50242	3.723498	0.831917	didntLike
12	63275	8.385879	1.669485	didntLike
13	5569	4.875435	0.728658	smallDoses
14	51052	4.680098	0.625224	didntLike
15	77372	15.299570	0.331351	didntLike

说明

序号	飞行公里数	玩游戏所占时间百分比	吃冰淇淋公升数	样本标签
1	40920	8.3	0.9	3
2	14488	7.1	1.6	2
3	26052	1.4	0.8	1
4	75136	13.1	0.42	1

https://github.com/Jack-Cherish/Machine-Learning/blob/master/kNN/2.%E6%B5%B7%E4%BC%A6%E7%BA%A6%E4%BC%9A/kNN_test02.py

KNN的深入探讨

- **Feature scaling很重要：**我们需要对输入数据做归一化（Normalization）
  - 防止某些属性（列）在计算相似度时起到“绝对权重”的情况

序号	飞行公里数	玩游戏所占时间百分比	吃冰淇淋公升数	样本标签
1	40920	8.3	0.9	3
2	14488	7.1	1.6	2
3	26052	1.4	0.8	1
4	75136	13.1	0.42	1

- **常见的归一化方法：数据标准化、Min-Max Scaling**

常见的归一化方法：数据标准化

1. 均值 $= \frac{\text{所有数据点的该维度值的总和}}{\text{数据点的数量}}$

2. 标准差 $= \sqrt{\frac{\sum_{i=1}^{n}(x_i - \text{均值})^2}{n}}$

3. 标准化值 $= \frac{x_i - \text{均值}}{\text{标准差}}$

假设有以下10个三维(x,y,z)数据点

1. (2, 3, 4)
2. (5, 7, 8)
3. (1, 6, 9)
4. (4, 2, 5)
5. (3, 5, 2)
6. (8, 1, 3)
7. (7, 4, 6)
8. (6, 8, 7)
9. (9, 10, 11)
10. (5, 3, 12)

每个维度的均值

x 维度的均值：5.0
y 维度的均值：4.9
z 维度的均值：6.7

每个维度的标准差

x 维度的标准差：2.449
y 维度的标准差：2.700
z 维度的标准差：3.164

标准化之后的数据

1. (-1.225, -0.704, -0.853)
2. (0.000, 0.778, 0.411)
3. (-1.633, 0.407, 0.727)
4. (-0.408, -1.074, -0.537)
5. (-0.816, 0.037, -1.486)
6. (1.225, -1.444, -1.169)
7. (0.816, -0.333, -0.221)
8. (0.408, 1.148, 0.095)
9. (1.633, 1.889, 1.359)
10. (0.000, -0.704, 1.675)

标准化后的数据点现在具有均值为0和标准差为1的特性，这使得它们更适合某些机器学习算法和统计分析

常见的归一化方法： Min-Max缩放

$$\text{归一化值} = \frac{x_i - \text{最小值}}{\text{最大值} - \text{最小值}}$$

1. (45, 48, 65)
2. (68, 68, 10)
3. (84, 22, 37)
4. (88, 71, 89)
5. (89, 13, 59)
6. (66, 40, 88)
7. (47, 89, 82)
8. (38, 26, 78)
9. (73, 10, 21)
10. (81, 70, 80)

x 维度（第一个坐标）的最小值：38，最大值：89

y 维度（第二个坐标）的最小值：10，最大值：89

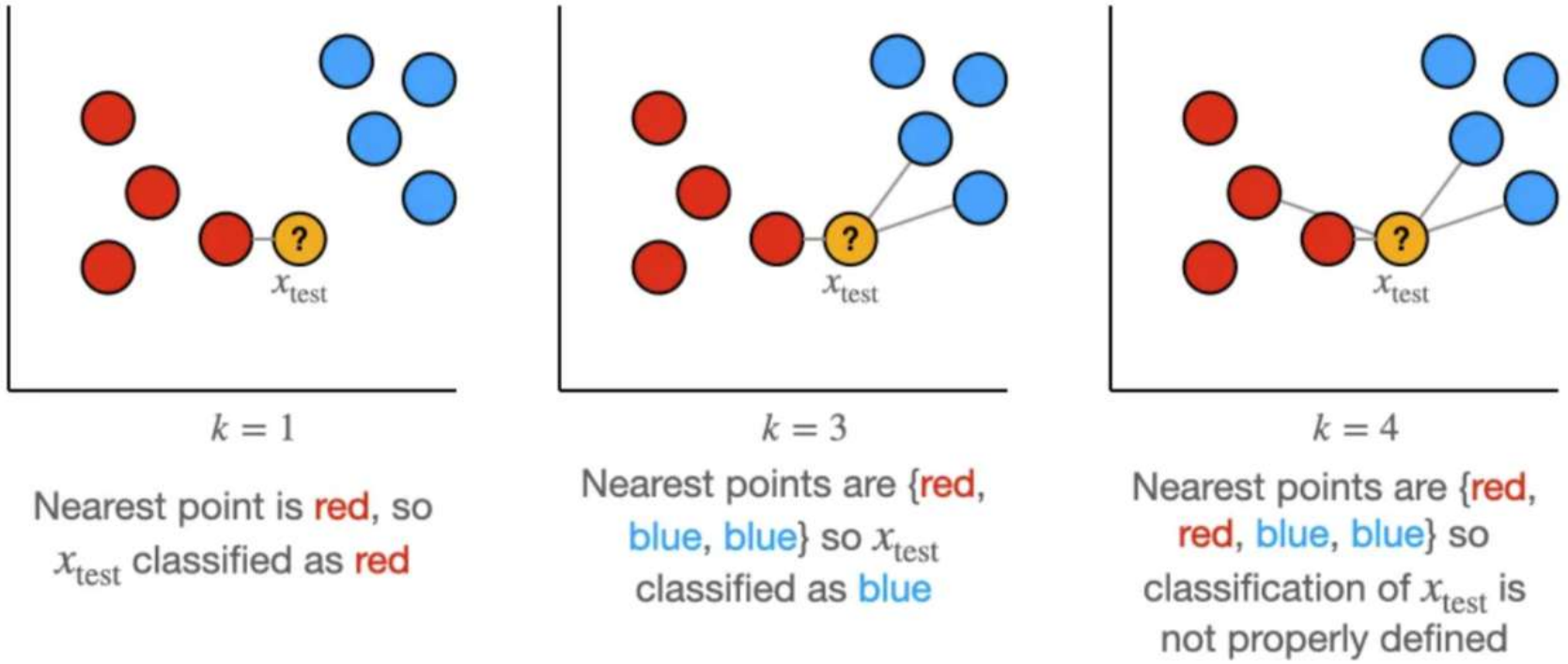
z 维度（第三个坐标）的最小值：10，最大值：89

归一化后的数据点都被缩放到0到1的范围

1. (0.137, 0.481, 0.696)
2. (0.588, 0.734, 0.000)
3. (0.902, 0.152, 0.342)
4. (0.980, 0.772, 1.000)
5. (1.000, 0.038, 0.620)
6. (0.549, 0.380, 0.987)
7. (0.176, 1.000, 0.911)
8. (0.000, 0.203, 0.861)
9. (0.686, 0.000, 0.139)
10. (0.843, 0.759, 0.886)

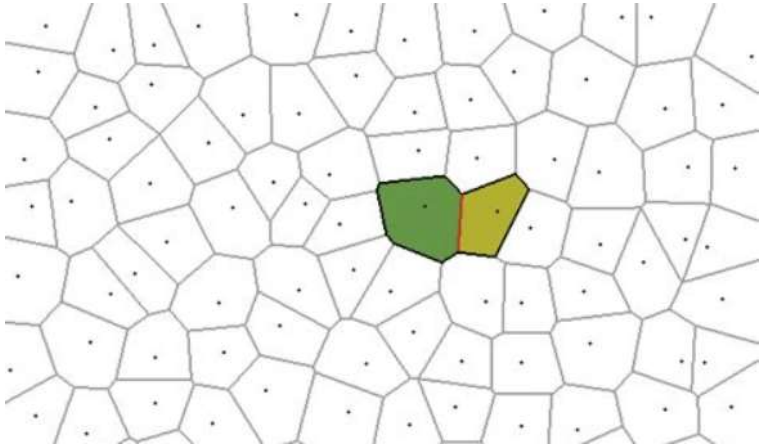
KNN的深入探讨

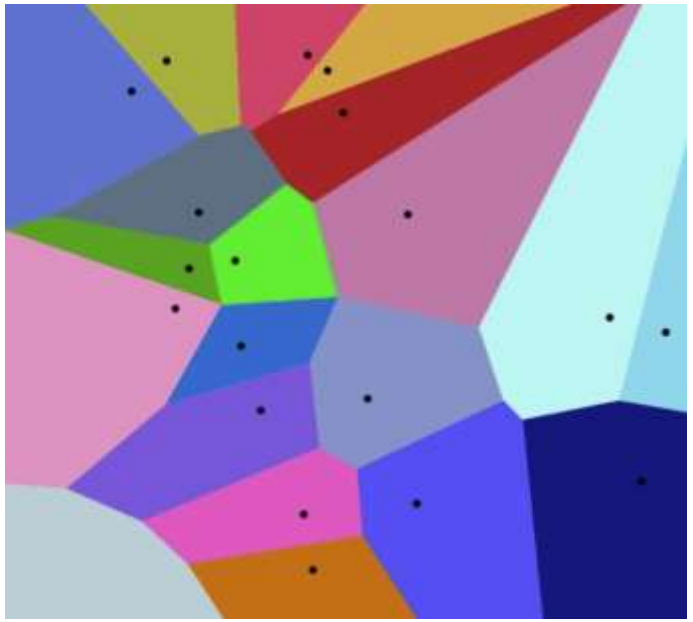
- 不同的K值会产生不同的分类结果
  - 如果是二分类的话，避免将K取为偶数



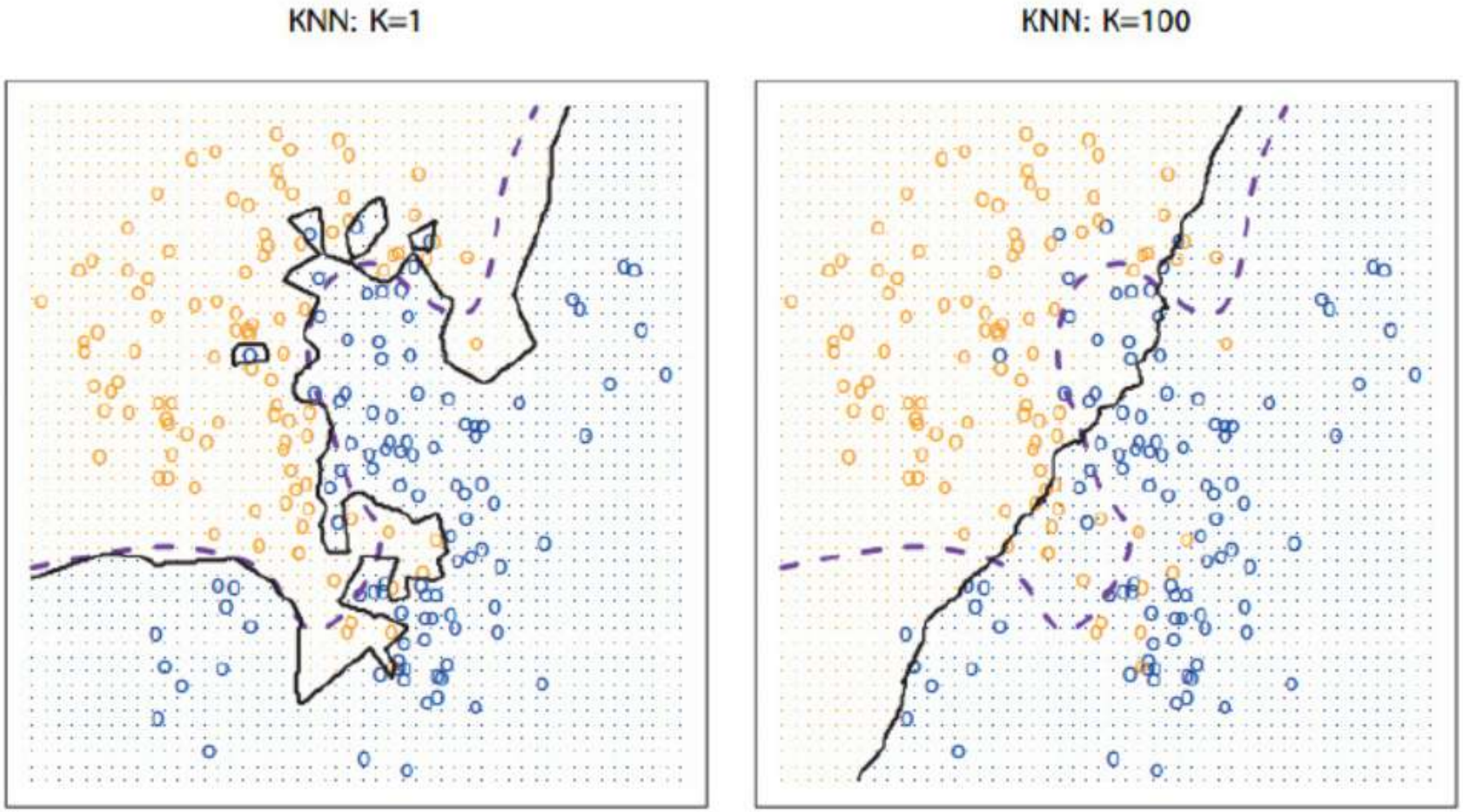
KNN的深入探讨

- **决策边界(decision boundaries)**
  - 维诺图(Voronoi diagram)，离某一数据点最近点的集合。图中绿色部分离其中的黑点最近，这一部分所有点离其他的黑点都比之远。图中的红色线是离两个黑点（叫site）同样远点组成的。



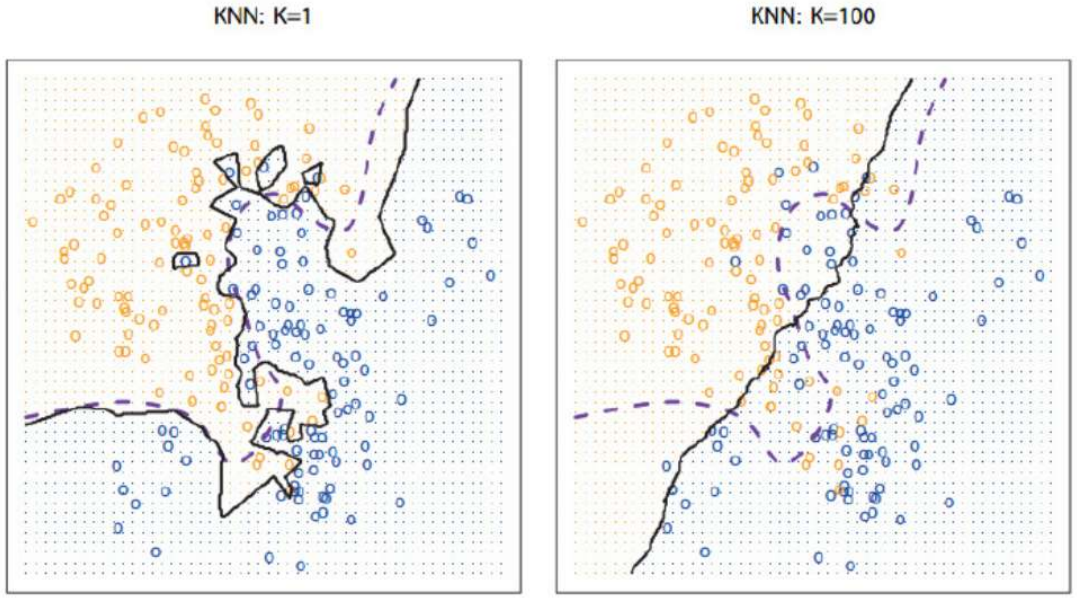


以下哪一个KNN得到的决策边界更好?



KNN的深入探讨

- **决策边界(decision boundaries)**
  - 随着大量样本和可能的标签噪声被引入，决策边界可能会变得差
  - 更大的*k*会产生更平滑的边界效果
  - 当*k*等于样本总数*N*时，总是预测多数类



KNN的应用：手写识别

- 训练集：手写数字0-9，共180个。测试数据88个
  - K=3

```python
# k-近邻算法
def classify0(inX, dataSet, labels, k):  # 输入向量，训练数据，标签，参数k
    dataSetSize = dataSet.shape[0]  # 数据个数
    diffMat = tile(inX, (dataSetSize, 1)) - dataSet  # tile函数，求输入数据与训练数据对应值的相减
    sqDiffMat = diffMat ** 2  # 平方
    sqDistances = sqDiffMat.sum(axis=1)  # 求和
    distances = sqDistances ** 0.5  # 开根号
    sortedDistIndicies = distances.argsort()  # 返回数组值从小到大的索引
    classCount = {}  # 创建一个字典，用于记录每个实例对应的频数
    for i in range(k):
        voteIlabel = labels[sortedDistIndicies[i]]  # 选择k个距离最小的点，对应标签
        classCount[voteIlabel] = classCount.get(voteIlabel, 0) + 1  # 统计频数
    sortedClassCount = sorted(classCount.items(), key=operator.itemgetter(1), reverse=True)  # 排序，reverse=True降序
    # python2中用iteritems
    return sortedClassCount[0][0]  # 返回最多的
```

```
Run:  03_kNN_handwriting
预测结果为：9，真实值为：9
预测结果为：9，真实值为：9
预测结果为：9，真实值为：9
预测结果为：9，真实值为：9
预测结果为：9，真实值为：9
预测错误的总数为：10
手写数字识别系统的错误率为：0.010571

Process finished with exit code 0
```

0_0.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

```
00000000000001111000000000000000
00000000000111111100000000000000
00000000011111111110000000000000
00000011111111111111100000000000
00000001111111101111110000000000
00000011111110000011110000000000
00000011111110000000111000000000
00000011111110000000111000000000
00000011111110000000111000000000
00000011111110000000011100000000
00000011111110000000011100000000
00000011111110000000011100
```

K最近邻算法

KNN的应用：图像分类

- CIFAR10数据集，60000张图像
- 10个类别，图像分辨率为32*32
- 50000张为训练集，10000张为测试集
- 选取500张进行测试，k=3

```
from cs231n.classifiers import KNearestNeighbor

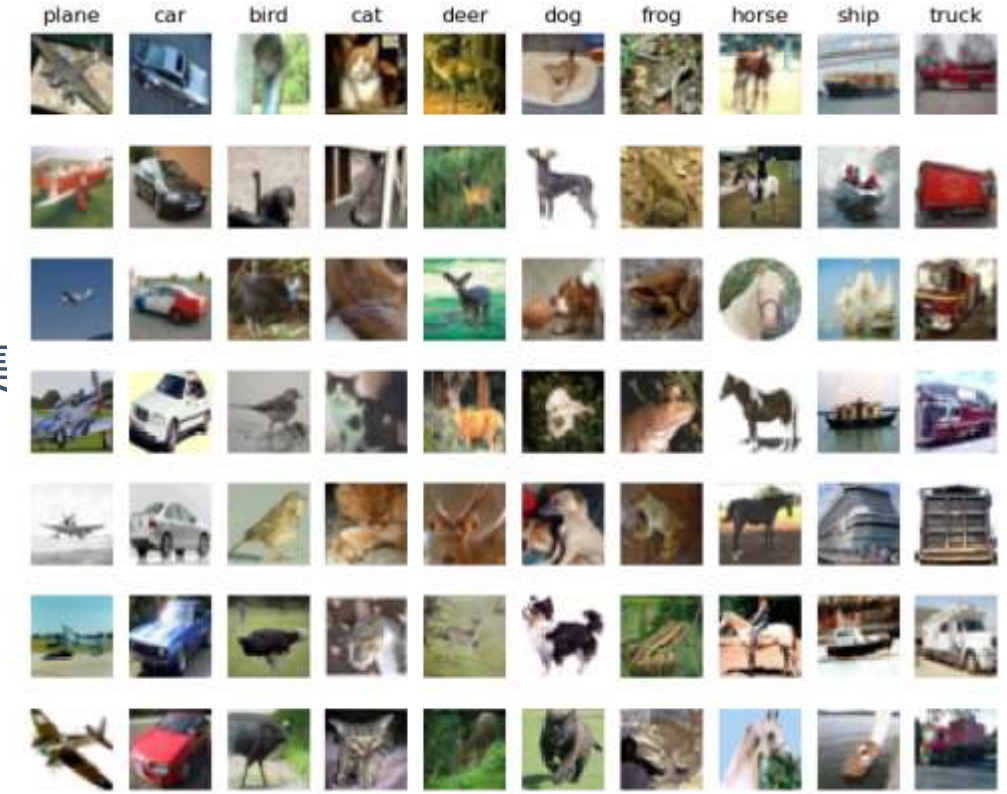
# Create a kNN classifier instance.
# Remember that training a kNN classifier is a noop:
# the Classifier simply remembers the data and does no further processing
classifier = KNearestNeighbor()
classifier.train(X_train, y_train)

# Open cs231n/classifiers/k_nearest_neighbor.py and implement
# compute_distances_two_loops.

# Test your implementation:
dists = classifier.compute_distances_two_loops(X_test)
# print(dists.shape)


# Now implement the function predict_labels and run the code below:
# We use k = 1 (which is Nearest Neighbor).
y_test_pred = classifier.predict_labels(dists, k=3)

# Compute and print the fraction of correctly predicted examples
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

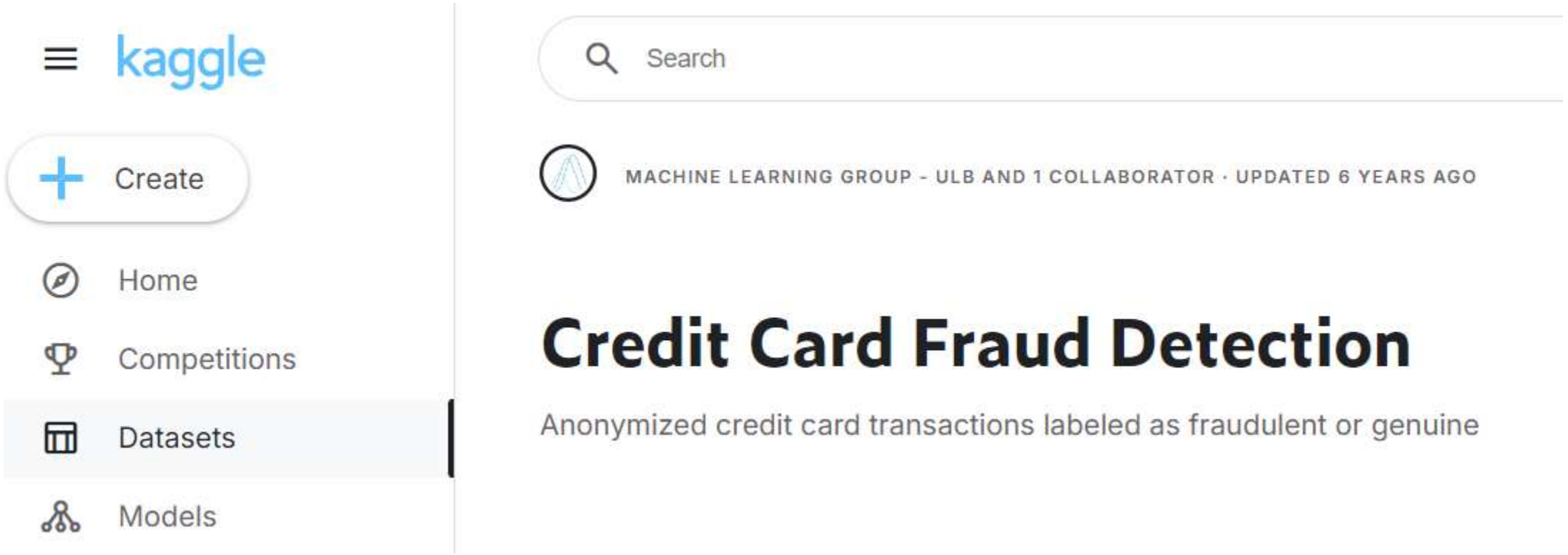



```
04_kNN_CIFAR10
D:\Anaconda3\python.exe E:/03_教学/SE_Machine_Le
Training data shape:  (50000, 32, 32, 3)
Training labels shape:  (50000,)
Test data shape:  (10000, 32, 32, 3)
Test labels shape:  (10000,)
Sampled data size: (5000, 3072) (500, 3072)
Got 57 / 500 correct => accuracy: 0.114000

Process finished with exit code 0
```

KNN的应用实例：银行信用卡欺诈检测

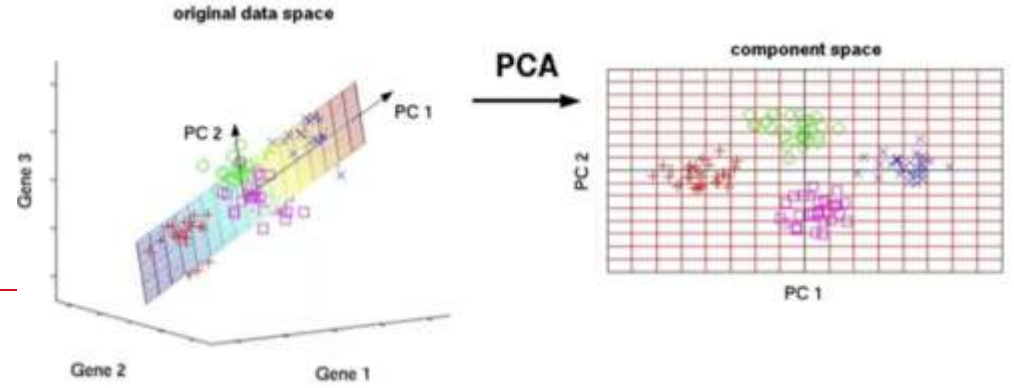
- 数据来自Kaggle，字段已经经过脱敏及预处理



KNN的应用实例：银行信用卡欺诈检测

关于数据的描述：

- **数据集包含了2013年9月欧洲持卡人使用信用卡进行的交易记录。**
- 这个数据集展示了两天内的交易情况，**其中有492笔欺诈交易，总共284,807笔交易。**数据集存在**严重的不平衡问题，正类（欺诈）仅占所有交易的0.172%。**
- 数据集仅包含数值型输入变量，这些变量是PCA变换的结果。由于保密问题，无法提供原始特征和更多关于数据的背景信息。**特征V1、V2、...、V28是通过PCA获得的主成分，只有'Time'和'Amount'这两个特征没有经过PCA变换。**特征'Time'表示每笔交易与数据集中的第一笔交易之间的时间差，以秒为单位。特征'Amount'表示交易金额，可用于示例依赖的成本敏感学习。特征'Class'是响应变量，欺诈为1，非欺诈为0。



KNN的应用实例：银行信用卡欺诈检测

数据集的信息

- 由于数据集比较大，计算最近邻会花费大量时间，所以只取了11658条记录，其中欺诈记录为49条

数据集前几行：

	Time	V1	V2	V3	...	V27	V28	Amount	Class
0	0.0	-1.359807	-0.072781	2.536347	...	0.133558	-0.021053	149.62	0
1	0.0	1.191857	0.266151	0.166480	...	-0.008983	0.014724	2.69	0
2	1.0	-1.358354	-1.340163	1.773209	...	-0.055353	-0.059752	378.66	0
3	1.0	-0.966272	-0.185226	1.792993	...	0.062723	0.061458	123.50	0
4	2.0	-1.158233	0.877737	1.548718	...	0.219422	0.215153	69.99	0

数据集基本信息：

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11658 entries, 0 to 11657
Data columns (total 31 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Time    11658 non-null  int64
 1   V1      11658 non-null  float64
 2   V2      11658 non-null  float64
 3   V3      11658 non-null  float64
 4   V4      11658 non-null  float64
 5   V5      11658 non-null  float64
 6   V6      11658 non-null  float64
 7   V7      11658 non-null  float64
 8   V8      11658 non-null  float64
 9   V9      11658 non-null  float64
 10  V10     11658 non-null  float64
```

KNN的应用实例：银行信用卡欺诈检测

以欺诈记录中的10条作为测试集

```python
# 导入所需的库
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, accura

# 加载数据集
# 这里假设数据集已下载并保存为 'creditcard.csv'
df = pd.read_csv('creditcard.csv')

# 查看数据集基本信息
print("数据集基本信息：")
print(df.info())

# 查看前几行数据
print("\n数据集前几行：")
print(df.head())

# 筛选出欺诈记录
fraud_df = df[df['Class'] == 1]
non_fraud_df = df[df['Class'] == 0]

# 随机选择10条欺诈记录作为测试集
fraud_test_df = fraud_df.sample(n=10, random_state=42)
```

```python
# 剩余的欺诈记录和所有非欺诈记录作为训练集
fraud_train_df = fraud_df.drop(fraud_test_df.index)
train_df = pd.concat([fraud_train_df, non_fraud_df])

print("\n测试集：")
print(fraud_test_df)

# 特征和标签
X_train = train_df.drop(['Class'], axis=1)
y_train = train_df['Class']
X_test = fraud_test_df.drop(['Class'], axis=1)
y_test = fraud_test_df['Class']
```

KNN的应用实例：银行信用卡欺诈检测

KNN预测

```
# Step 2: 标准化特征数据
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)  # 对训练数据进行标准化
X_test = scaler.transform(X_test)  # 对测试数据进行相同的变换

# Step 3: 训练 KNN 模型
k = 5  # 可以调整 k 值，选择合适的 k
knn = KNeighborsClassifier(n_neighbors=k)
knn.fit(X_train, y_train)  # 训练模型

# Step 4: 预测测试集
y_pred = knn.predict(X_test)
y_prob = knn.predict_proba(X_test)[:, 1]  # 获取预测的概率值

# Step 5: 输出预测结果
# 将 X_test 转换为 DataFrame 以获取列名
X_test_df = pd.DataFrame(X_test, columns=train_df.drop(['Class'], axis=1).columns)

# 创建包含预测结果的 DataFrame
predicted_fraud_df = X_test_df.copy()
predicted_fraud_df['ActualClass'] = y_test.values
predicted_fraud_df['PredictedClass'] = y_pred
predicted_fraud_df['PredictionProbability'] = y_prob

print("\n测试集中的预测结果：")
print(predicted_fraud_df)
```

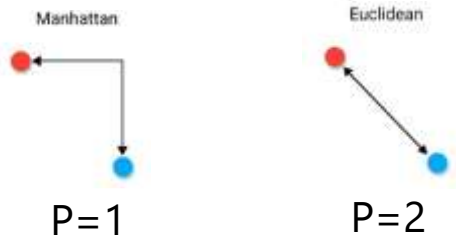
KNeighborsClassifier

class sklearn.neighbors.KNeighborsClassifier(*n_neighbors=5*, ***, *weights='uniform'*, *algorithm='auto'*, *leaf_size=30*, *p=2*, *metric='minkowski'*, *metric_params=None*, *n_jobs=None*) [source]

Classifier implementing the k-nearest neighbors vote.

Read more in the User Guide.

$$d(x,y) = \left(\sum_{i=1}^{n} |x_i - y_i|^p\right)^{\frac{1}{p}}$$




```
# Step 6: 输出测试集中的结果
test_results_df = pd.DataFrame({
    'TransactionAmount': X_test[:, 0],
    'TransactionTime': X_test[:, 1],
    'ActualClass': y_test,
    'PredictedClass': y_pred,
    'PredictionProbability': y_prob
})

# Step 7: 评估模型性能
accuracy = accuracy_score(y_test, y_pred)
print("\n模型的准确性：{:.2f}%".format(accuracy * 100))
```

KNN的应用实例：银行信用卡欺诈检测

K=5时测试集的预测结果：准确率80%

测试集：

	Time	V1	V2	...	V28	Amount	Class
6609	8090	-1.783229	3.402794	...	0.243746	1.00	1
10801	18399	-14.474437	6.503185	...	-0.992916	1.00	1
10897	18690	-15.398845	7.472324	...	-1.178063	1.00	1
10690	18088	-12.224021	3.854150	...	-0.775036	1218.89	1
6734	8451	0.314597	2.660670	...	0.185030	1.00	1
8335	11131	-1.426623	4.141986	...	0.711607	1.00	1
8312	11092	0.378275	3.914797	...	0.411910	1.00	1
8296	11080	-2.125490	5.973556	...	1.108933	1.00	1
8845	12095	-4.727713	3.044469	...	0.705698	30.39	1
6820	8614	-2.169929	3.639654	...	0.298954	1.00	1

[10 rows x 33 columns]

模型的准确性：80.00%

测试集中的预测结果：

	Time	V1	...	PredictedClass	PredictionProbability
0	0.067202	-1.014587	...	1	1.0
1	1.797287	-9.228799	...	1	1.0
2	1.846123	-9.827109	...	1	1.0
3	1.745094	-7.772248	...	0	0.0
4	0.127786	0.343202	...	1	1.0
5	0.577551	-0.783779	...	0	0.2
6	0.571006	0.384416	...	1	1.0
7	0.568992	-1.236111	...	1	0.8
8	0.739332	-2.920364	...	1	1.0
9	0.155141	-1.264874	...	1	1.0

KNN的应用实例：银行信用卡欺诈检测

如果我们改变K值呢?

- K=3时，80%，
- K=11时，90%
- K=19时，100%

测试集中的预测结果：

	Time	V1	...	PredictedClass	PredictionProbability
0	0.067202	-1.014587	...	1	0.909091
1	1.797287	-9.228799	...	1	0.727273
2	1.846123	-9.827109	...	1	0.727273
3	1.745094	-7.772248	...	0	0.272727
4	0.127786	0.343202	...	1	1.000000
5	0.577551	-0.783779	...	1	0.545455
6	0.571006	0.384416	...	1	1.000000
7	0.568992	-1.236111	...	1	0.545455
8	0.739332	-2.920364	...	1	1.000000
9	0.155141	-1.264874	...	1	1.000000

[10 rows x 33 columns]

模型的准确性：90.00%

测试集中的预测结果：

	Time	V1	...	PredictedClass	PredictionProbability
0	0.067202	-1.014587	...	1	0.842105
1	1.797287	-9.228799	...	1	0.736842
2	1.846123	-9.827109	...	1	0.736842
3	1.745094	-7.772248	...	1	0.526316
4	0.127786	0.343202	...	1	0.842105
5	0.577551	-0.783779	...	1	0.736842
6	0.571006	0.384416	...	1	0.684211
7	0.568992	-1.236111	...	1	0.736842
8	0.739332	-2.920364	...	1	0.842105
9	0.155141	-1.264874	...	1	0.842105

[10 rows x 33 columns]

模型的准确性：100.00%

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

目 录



1 K最近邻算法（KNN）

2 协同过滤（CF）

3 Spark Mllib示例

协同过滤（CF）

- 推荐系统（Recommender Systems）是解决信息过载的有效手段



“People read around 10 MB worth of material a day, hear 400 MB a day, and see 1 MB of information every second” - The Economist, November 2006

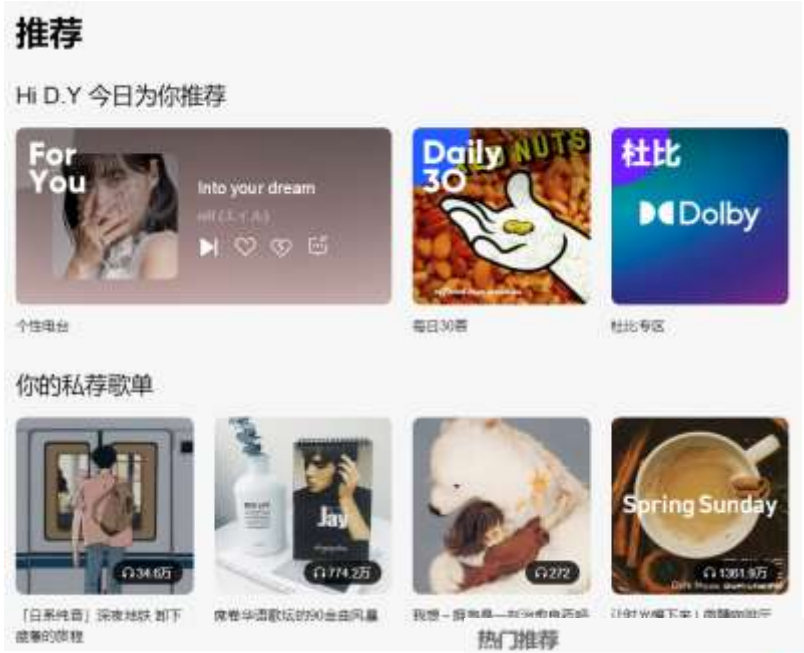
- 搜索 vs. 推荐





协同过滤（CF）

推荐系统的应用广泛



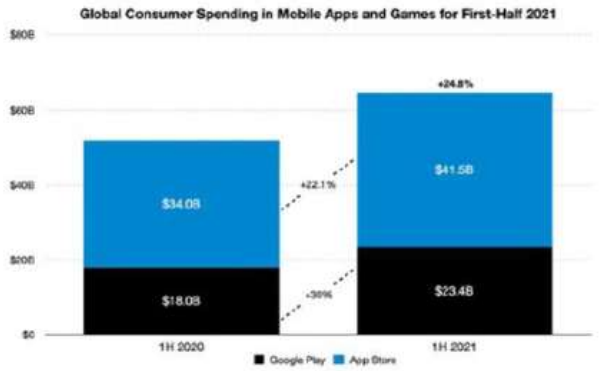




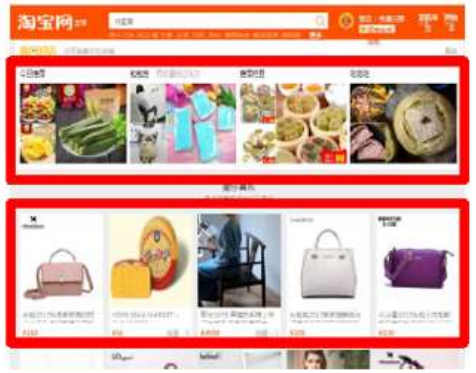
Display ad on YouTube



推荐技术是商业化最成功的AI技术



2021H1, App Store收入415亿美元，Google Play 234亿美元；



阿里广告收入超1745亿人民币（2019年）



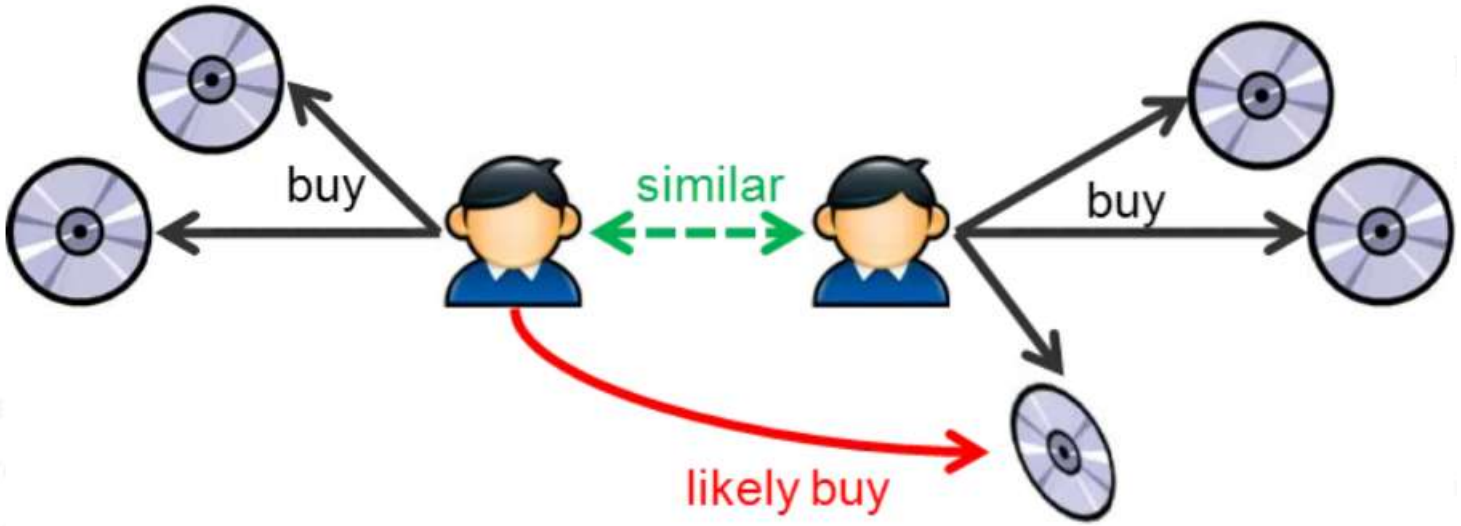
Google广告收入2297亿美元（2018年）

2018年天猫双十一达成2135亿元交易，背后**产生了453亿次AI个性化推荐**，使消费者在海量商品中**更容易找到真正需要**的商品

麦肯锡（McKinsey）的数据显示，亚马逊上35%的购买行为是其推荐系统的结果；用户在YouTube 上观看视频的时间中有 70% 来自推荐；人们在Netflix上观看的内容中有75%来自推荐

协同过滤（Collaborative Filtering, CF）是推荐系统最重要的技术

- 基于很直觉的假设，即兴趣（购物行为）相似的用户他们很可能会购买类似的商品



协同过滤（CF）

- 用户对物品的交互可以用效用矩阵（utility matrix）或叫观测矩阵来表示
  - 分为显示和隐式 (explicit or implicit).

	Flash	Arrow	Spiderman	Batman	Supergirl
User1	1	5			2
User2		5			
User3				3	
User4	1		4	3	

4×5 utility matrix for superhero ratings

- 协同过滤分两种:
  - 基于用户的协同过滤User-user collaborative filtering
  - 基于物品的协同过滤Item-item collaborative filtering

物以类聚
人以群分

协同过滤（CF）

以基于用户的协同过滤为例

对于用户 x

- *Step 1:* 找到 N个用户，他们和 x 的行为是相似的

依据什么来找“相似”

- *Step 2:* 基于其他N个用户的行为来估计x对未交互过物品的评分

用什么方式来估计

寻找 “相似” 用户 （搜索k-最近邻）

- 使 r_x 和 r_y 分别表示用户x 和 y 的评分
- 计算余弦相似度

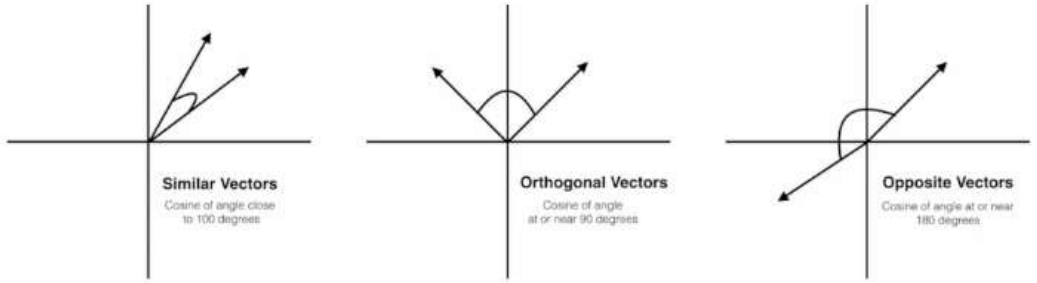
$$\text{sim}(x, y) = \cos(r_x, r_y) = \frac{r_x \cdot r_y}{||r_x|| \cdot ||r_y||}$$

r_x = [1, _, _, 1, 3]
r_y = [1, _, 2, 2, _]

r_x, r_y as points:
r_x = {1, 0, 0, 1, 3}
r_y = {1, 0, 2, 2, 0}

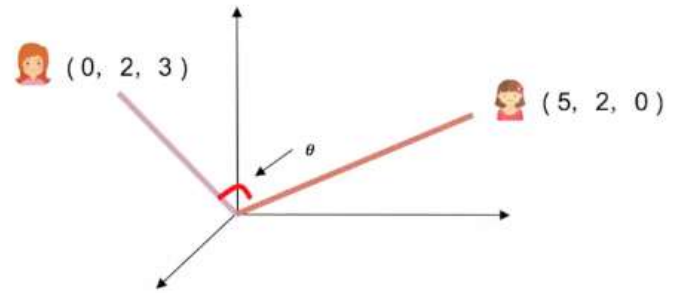
Let's practice!

θ	30 degrees	90 degrees	150 degrees
cosθ	0.86	0	-0.86
Are two vectors similar ?	Similar	Fair / Okay / Might be not	Not Similar



Formular:

$$\cos(u_i, u_k) = \frac{\sum_{j=1}^{m} v_{ij} v_{kj}}{\sqrt{\sum_{j=1}^{m} v_{ij}^2 \sum_{j=1}^{m} v_{kj}^2}}$$



Result:

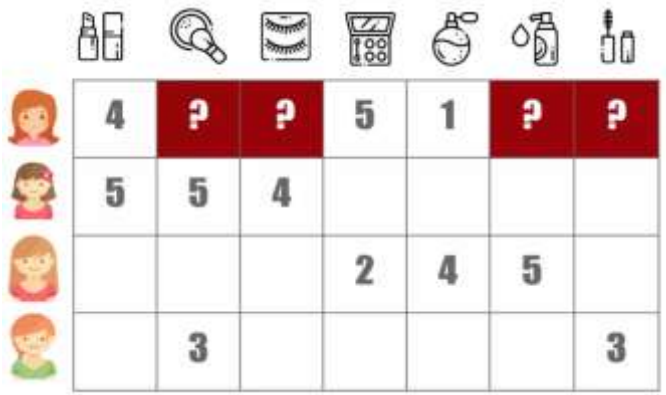
cos() = $\frac{0*5+2*2+3*0}{\text{sqrt}(0\text{^}2+2\text{^}2+3\text{^}2) * \text{sqrt}(5\text{^}2+2\text{^}2+0\text{^}2)}$ = **0.21**

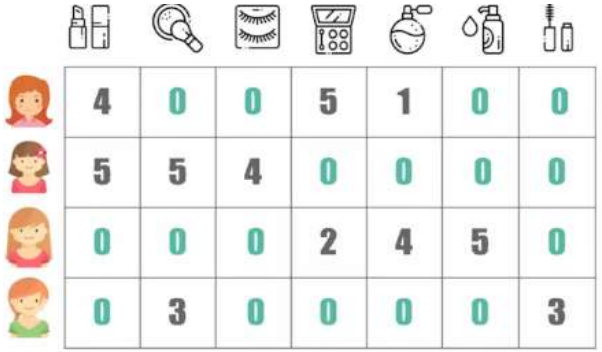
估计评分

- $r_{xi} = \frac{1}{k} \sum_{y \in N} r_{yi}$
- 或者更进一步，加入相似度系数:
  - $r_{xi} = \frac{\sum_{y \in N} s_{xy} r_{yi}}{\sum_{y \in N} s_{xy}}$, where $s_{xy} = sim(x, y)$

协同过滤（CF）

Case





P1&P2:
Cosine similarity = **0.38**
(4*5 + 0*5 + 0*4 + 5*0 + 1*0 + 0*0 + 0*0) /
[(SQRT(4^2+0^2+ 0^2+ 5^2+1^2+0^2+0^2)*
(SQRT(5^2+5^2+ 4^2+ 0^2+0^2+0^2+0^2)]

P1&P3:
Cosine similarity = **0.32**
(4*0 + 0*0 + 0*0 + 5*2 + 1*4 + 0*5 + 0*0) /
[(SQRT(4^2+0^2+ 0^2+ 5^2+1^2+0^2+0^2)*
(SQRT(0^2+0^2+ 0^2+ 2^2+4^2+5^2+0^2)]

P1&P4:
Cosine similarity = **0.0**
(4*0 + 0*3 + 0*0 + 5*0 + 1*0 + 0*0 + 0*3) /
[(SQRT(4^2+0^2+ 0^2+ 5^2+1^2+0^2+0^2)*
(SQRT(0^2+3^2+ 0^2+ 0^2+0^2+0^2+3^2)]

1 Assign average score to each person

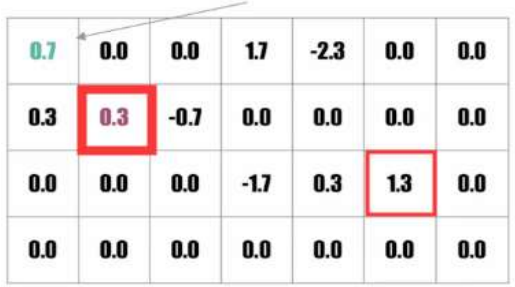
Person 1: average score = (4+5+1) = 10/3



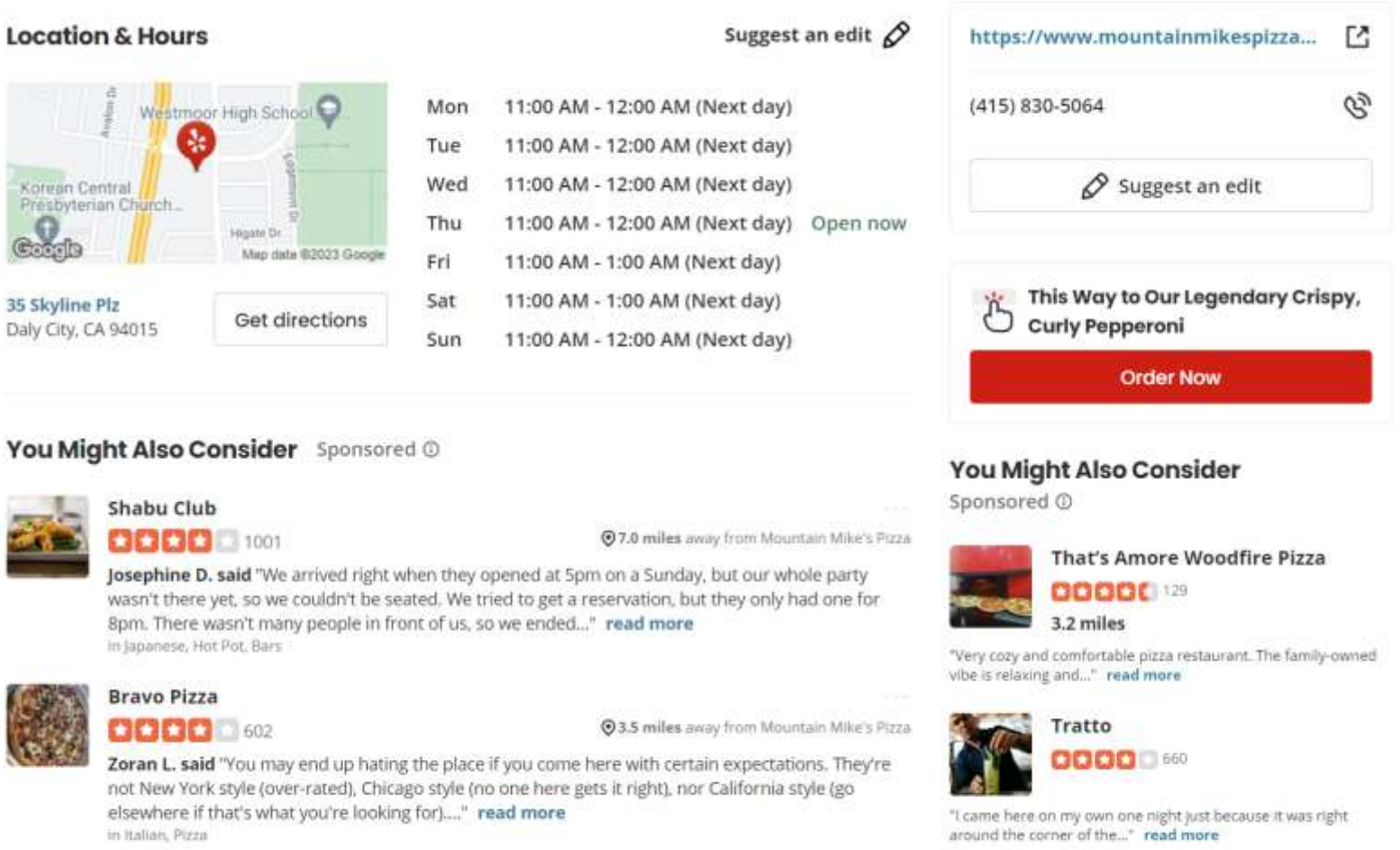
2 Normalize

Formular: Score - Average(Scores)

Eg: Person 1 score: 4 - 10/3 = 0.7



如果是这样的真实数据会怎么做？Yelp dataset



协同过滤（CF）

如果是这样的真实数据应该怎么做？Yelp dataset

yelp Dataset

Dataset Documentation

Documentation

Main

Photos

FAQ

Yelp Dataset JSON

Each file is composed of a single object type, one JSON-object per-line.

Take a look at some examples to get you started: https://github.com/Yelp/dataset-examples

Note: the follow examples contain inline comments, which are technically not valid JSON. This is done here to simplify the documentation and explaining the structure, the JSON files you download will not contain any comments and will be fully valid JSON.

business.json

Contains business data including location data, attributes, and categories.

```
// integer, 0 or 1 for closed or open, respecti
"is_open": 1,

// object, business attributes to values. note:
"attributes": {
    "RestaurantsTakeOut": true,
    "BusinessParking": {
        "garage": false,
        "street": true,
        "validated": false,
        "lot": false,
        "valet": false
    },
},

// an array of strings of business categories
"categories": [
    "Mexican",
    "Burgers",
    "Gastropubs"
],

// an object of key day to value hours, hours a
"hours": {
    "Monday": "10:00-21:00",
    "Tuesday": "10:00-21:00",
    "Friday": "10:00-21:00",
    "Wednesday": "10:00-21:00",
    "Thursday": "10:00-21:00",
    "Sunday": "11:00-18:00",
    "Saturday": "10:00-21:00"
}
}
```

```
{
    // string, 22 character unique string b
    "business_id": "tnhfDv5Il8EaGSXZGiuQGg",

    // string, the business's name
    "name": "Garaje",

    // string, the full address of the busi
    "address": "475 3rd St",

    // string, the city
    "city": "San Francisco",

    // string, 2 character state code, if a
    "state": "CA",

    // string, the postal code
    "postal code": "94107",

    // float, latitude
    "latitude": 37.7817529521,

    // float, longitude
    "longitude": -122.39612197,

    // float, star rating, rounded to half-
    "stars": 4.5,

    // integer, number of reviews
    "review_count": 1198,
```

review.json

Contains full review text data including the user_id that wr
the review is written for.

```
{
    // string, 22 character unique review id
    "review_id": "zdSx_SD6obEhz9VrW9uAWA",

    // string, 22 character unique user id, m
    "user_id": "Ha3iJu77CxlrFm-vQRs_8g",

    // string, 22 character business id, maps
    "business_id": "tnhfDv5Il8EaGSXZGiuQGg",

    // integer, star rating
    "stars": 4,

    // string, date formatted YYYY-MM-DD
    "date": "2016-03-09",

    // string, the review itself
    "text": "Great place to hang out after wor

    // integer, number of useful votes receive
    "useful": 0,

    // integer, number of funny votes receive
    "funny": 0,

    // integer, number of cool votes received
    "cool": 0
}
```

user.json

User data including the user's friend mapping and all the metadata associated with the user.

```
{
    // string, 22 character unique user id, maps to the user in user.json
    "user_id": "Ha3iJu77CxlrFm-vQRs_8g",

    // string, the user's first name
    "name": "Sebastien",

    // integer, the number of reviews they've written
    "review_count": 56,

    // string, when the user joined Yelp, fo
    "yelping_since": "2011-01-01",

    // array of strings, an array of the use
    "friends": [
        "wqoXYLWmpkEH0YvTmHBsJQ",
        "KUXLLiJGrjtSsapmxmpvTA",
        "6e9rJKQC3n0RSKyHLViL-Q"
    ],

    // integer, number of useful votes sent
    "useful": 21,

    // integer, number of funny votes sent b
    "funny": 88,

    // integer, number of cool votes sent by
    "cool": 15,

    // integer, number of fans the user has
    "fans": 1032,

    // float, average rating of all reviews
    "average_stars": 4.31,

    // integer, number of hot compliments received by the user
    "compliment_hot": 339,

    // integer, number of more compliments received by the user
    "compliment_more": 668,

    // integer, number of profile compliments received by the user
    "compliment_profile": 42,

    // integer, number of cute compliments received by the user
    "compliment_cute": 62,

    // integer, number of list compliments received by the user
    "compliment_list": 37,

    // integer, number of note compliments received by the user
    "compliment_note": 356,

    // integer, number of plain compliments received by the user
    "compliment_plain": 68,

    // integer, number of cool compliments received by the user
    "compliment_cool": 91,

    // integer, number of funny compliments received by the user
    "compliment_funny": 99,

    // integer, number of writer compliments received by the user
    "compliment_writer": 95,

    // integer, number of photo compliments received by the user
    "compliment_photos": 50
```

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

目 录



1 K最近邻算法（KNN）

2 协同过滤（CF）

3 Spark MIlib示例

Spark-KNN示例代码

- Spark Mllib本身并不直接提供KNN的实现
- 可以使用第三方库
- https://github.com/saurfang/spark-knn

KNNClassifier

```
//read in raw label and features
val training = MLUtils.loadLibSVMFile(sc, "data/mllib/sample_libsvm_data.txt").toDF()

val knn = new KNNClassifier()
  .setTopTreeSize(training.count().toInt / 500)
  .setK(10)

val knnModel = knn.fit(training)

val predicted = knnModel.transform(training)
```

KNNRegression

```
//read in raw label and features
val training = MLUtils.loadLibSVMFile(sc, "data/mllib/sample_libsvm_data.txt").toDF()

val knn = new KNNRegression()
  .setTopTreeSize(training.count().toInt / 500)
  .setK(10)

val knnModel = knn.fit(training)

val predicted = knnModel.transform(training)
```

Spark Mllib示例

Spark for Collaborative Filtering

- https://spark.apache.org/docs/3.5.0/ml-collaborative-filtering.html
- ***matrix factorization-based collaborative filtering (not user-user or item-item CF)***
- ***ALS optimization***
- ***这里用到了优化方法，我们在后面的课程继续讨论***

```
from pyspark.ml.evaluation import RegressionEvaluator
from pyspark.ml.recommendation import ALS
from pyspark.sql import Row

lines = spark.read.text("data/mllib/als/sample_movielens_ratings.txt").rdd
parts = lines.map(lambda row: row.value.split("::"))
ratingsRDD = parts.map(lambda p: Row(userId=int(p[0]), movieId=int(p[1]),
                                     rating=float(p[2]), timestamp=int(p[3])))
ratings = spark.createDataFrame(ratingsRDD)
(training, test) = ratings.randomSplit([0.8, 0.2])

# Build the recommendation model using ALS on the training data
# Note we set cold start strategy to 'drop' to ensure we don't get NaN evaluation metrics
als = ALS(maxIter=5, regParam=0.01, userCol="userId", itemCol="movieId", ratingCol="rating",
          coldStartStrategy="drop")
model = als.fit(training)

# Evaluate the model by computing the RMSE on the test data
predictions = model.transform(test)
evaluator = RegressionEvaluator(metricName="rmse", labelCol="rating",
                                predictionCol="prediction")
rmse = evaluator.evaluate(predictions)
print("Root-mean-square error = " + str(rmse))

# Generate top 10 movie recommendations for each user
userRecs = model.recommendForAllUsers(10)
# Generate top 10 user recommendations for each movie
movieRecs = model.recommendForAllItems(10)
# Generate top 10 movie recommendations for a specified set of users
users = ratings.select(als.getUserCol()).distinct().limit(3)
userSubsetRecs = model.recommendForUserSubset(users, 10)
# Generate top 10 user recommendations for a specified set of movies
movies = ratings.select(als.getItemCol()).distinct().limit(3)
movieSubSetRecs = model.recommendForItemSubset(movies, 10)
```

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

Thank You

